CENTRALIZED SYSTEM & NETWORK MONITORING

Implementation, Live Testing and Alert Validation

Prepared by	Joelle Kasabreh
Instructor	Dr. Jehad Hamamreh
Course	Systems and Networks Administration
Project environment	Windows 11 host
Date	30 October 2026



Cover figure. Original four-panel monitoring dashboard captured after successful configuration.

Abstract

This project presents the design and implementation of a small-scale system and network monitoring solution for a Windows workstation. Telegraf is used to gather OS statistics and output to the Prometheus metrics format. Prometheus scrapes the metrics endpoint and store time-series data. while Grafana displays four monitoring panels in real-time showing CPU, Memory, Disk and Network receive/send usage. An alert rule has been set up within Grafana that monitors the Windows C drive. Once the drive usage exceeds 90% during the evaluation window it will trigger into the Firing state. This has been verified by checking the endpoints locally, ensuring the Prometheus targets are healthy, confirming changes to the visualization, the alert rule is evaluated and a live demo is recorded. This solution has been implemented to monitor one Windows host. Future improvements could include virtual network topologies.

1. Introduction

Monitoring systems can let administrators know how resources are being used, services are running and if there are any capacity issues. Without a central view of our system, we may not notice that our disk is almost full or that there is pressure on our resources. In this project we focused on having a real end to end pipeline rather than a simulated dashboard. Our values will come from the workstation, our agent will expose these values, we will collect them into a time series database and have a web interface to show them.

Our chosen environment had open source software components that can be extended to additional hosts. A real world example is that our C: drive filled up almost completely, and when I first tried to save our dashboard in Grafana it gave an error saying the database or disk was full. This then became a good example of why it is useful to monitor our capacity as we were able to successfully save it after.

2. Project Objectives

- Collect real windows CPU, RAM, Disk and Network data.
- Publish metrics to an HTTP /metrics endpoint on the host.
- Set up Prometheus to monitor itself and the windows telemetry exporter.
- Create readable time-series panels with correct units and legends.
- Detect high C: disk utilization using a periodically evaluated Grafana alert.
- Verify live operation, preserve visual evidence and export dashboard configuration.

3. System Design and Data Flow

The system has been implemented as a single host with all 3 core components running on a windows operating system. This means the Telegraf agent can expose an HTTP metrics endpoint which Prometheus will fetch from at the set interval. Grafana can then query Prometheus as a data source. There is no need for Grafana to interact with Telegraf directly.

Stage	Implementation	Purpose
Metrics collection	Telegraf on Windows	Samples CPU, memory, disks, interfaces and system information
Metrics exposition	127.0.0.1:9273/metrics	Serves Prometheus-formatted metrics
Time-series storage	Prometheus on localhost:9090	Scrapes configured jobs and evaluates time-series queries
Visualization	Grafana on localhost:3000	Plots metrics and hosts managed alert rules

Logical data path: Windows OS → Telegraf (9273) → Prometheus (9090) → Grafana (3000) → dashboard / alert rules.

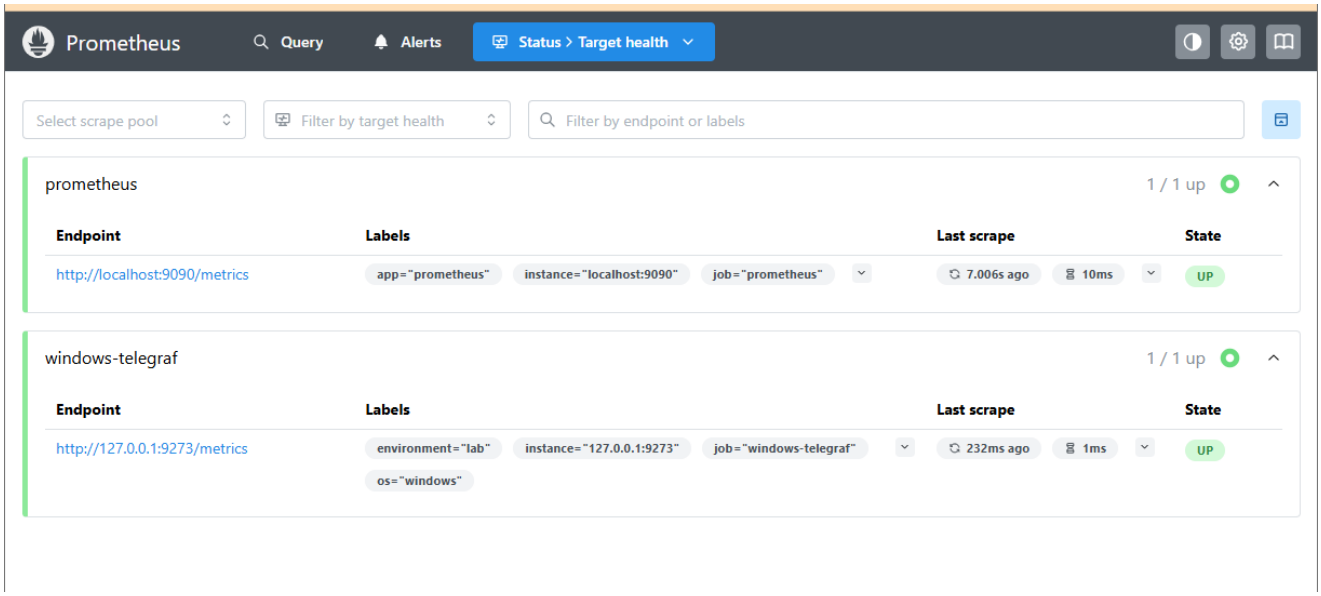


Figure 1. Prometheus Targets with both the internal service and windows-telegraf scrape job reporting UP.

4. Software and Configuration

Software	Role	Implementation note
Windows	Monitored operating system	Single actual production target for this exercise
Telegraf 1.40.2	Collection agent	Configured with cpu, disk, diskio, mem, net and system inputs
Prometheus 3.13.5	Scraper / time-series engine	Scrapes windows-telegraf and prometheus jobs
Grafana 13.3.0	Dashboard / alerting	Local dashboards and Grafana-managed rules
PromQL	Query language	Defines CPU, RAM, disk, network and alert expressions

4.1 Telegraf configuration

Step 1: Create a new folder in your local machine by extracting the Telegraf Windows archive. Create a simple monitoring.conf file containing only the following input plugins. inputs.cpu, inputs.disk, inputs.diskio, inputs.mem, inputs.net and inputs.system. Also create an output plugin for prometheus_client. This way we do not need to edit the huge example telegraf.conf.

Step 2: Run Telegraf in testing mode. This will show you the data being collected from the CPUs in the command prompt. This demonstrates that the host input plugins are working and can collect statistics from the operating system.

```
telegraf.exe --config monitoring.conf --test
```

Step 3: Now run Telegraf normally. We will keep the console open. This will show us that the input plugins are loading. It will also show us that the exporter is now listening on our local metrics endpoint.

We have created a separate monitoring.conf file within the Telegraf install directory. Our input plugins have been configured to collect data from the CPUs, Disk, Disk I/O, Memory, Network and System. We have also configured an output plugin for prometheus_client which will expose our metrics locally on port 9273 via TCP. We can confirm that the configuration file has been read by looking at the command line output.

```
telegraf.exe --config monitoring.conf
```

```
http://127.0.0.1:9273/metrics
```

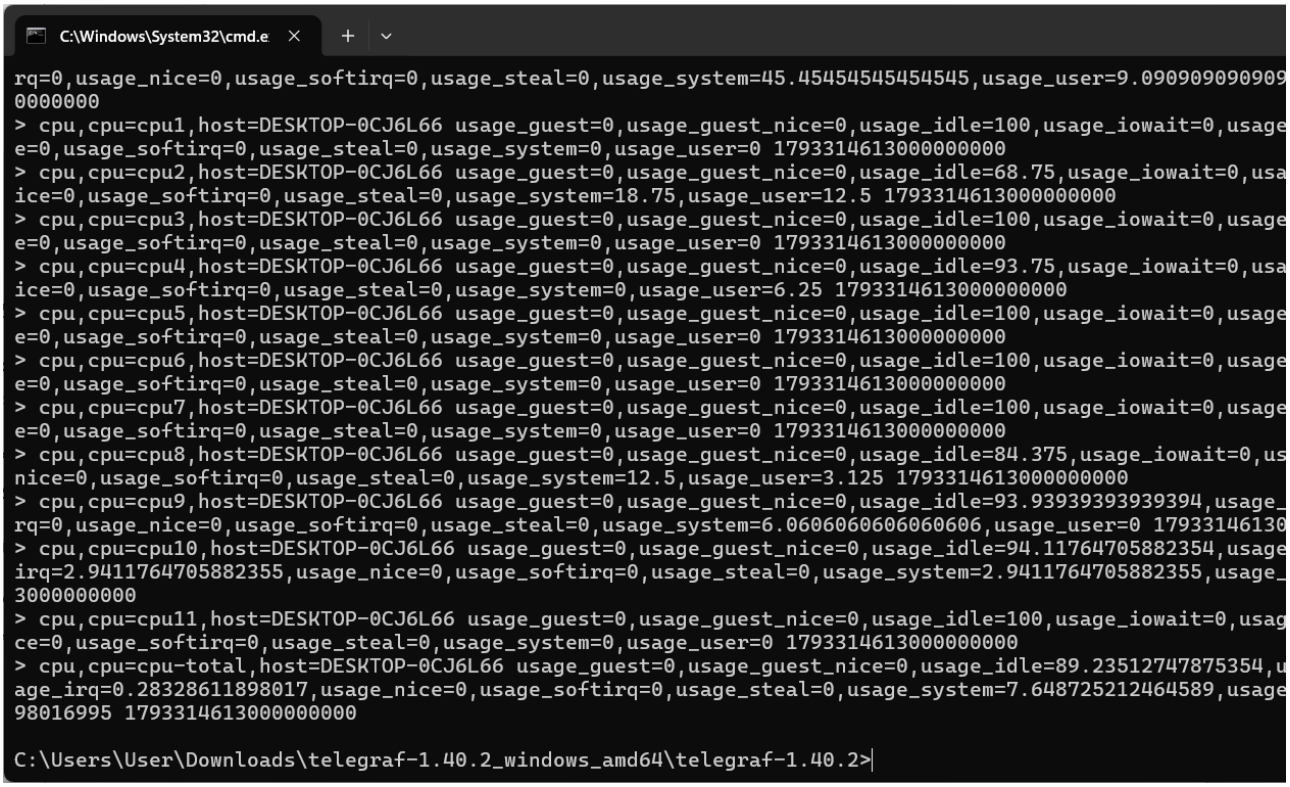


Figure 2. Command Prompt showing the Telegraf configuration test and CPU metrics output.

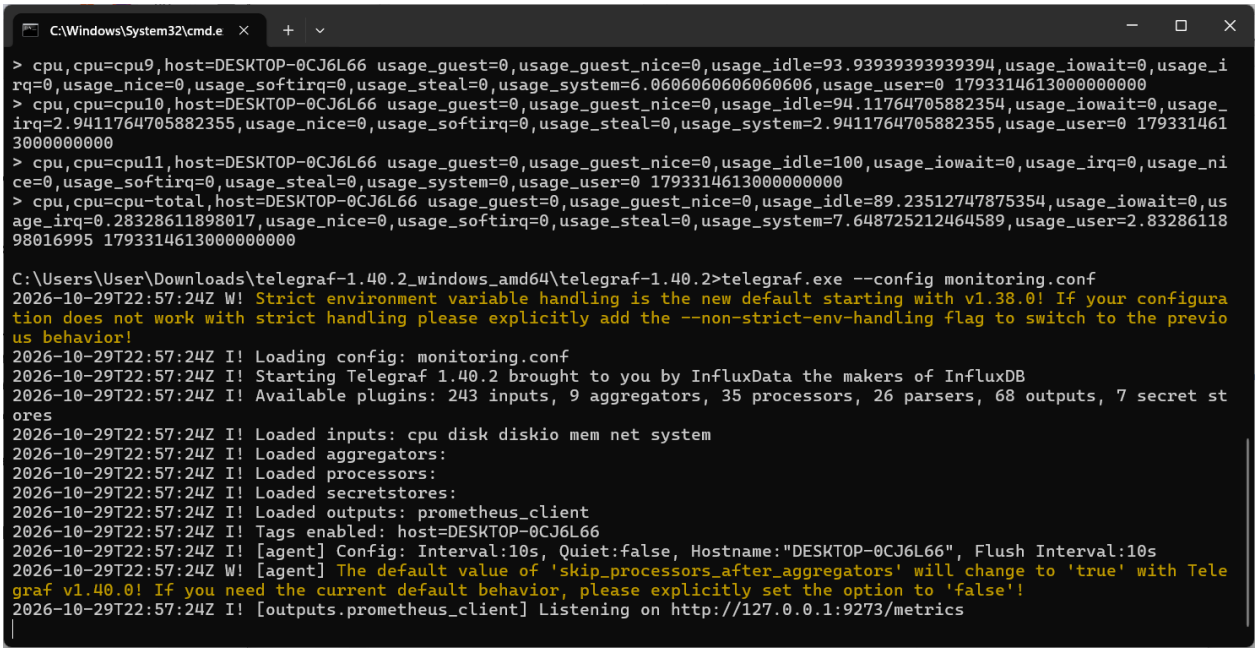


Figure 3. Command Prompt evidence: Telegraf startup, loaded input plugins, and the prometheus_client listener at 127.0.0.1:9273/metrics.

4.2 Prometheus configuration and checks

Step 4: We then edited the Prometheus scrape config to include our original prometheus job plus our windows-telegraf job. Within the scrape targets we pointed to the port Telegraf is listening on (9273).

Step 5: We used promtool to validate the YAML config file had been edited correctly. The result came back as SUCCESS meaning the config is correct.

Step 6: We ran our Prometheus service from command prompt. The logs show that our time series database has started and the web server is up and running. Both ends have now come up as UP on the browser target health page.

The Prometheus service was configured using the prometheus.yml file. The global scrape interval was set to 15 seconds. The windows-telegraf scrape job was set to collect metrics every 10 seconds. We added some labels to help identify what data we are collecting (os="windows", environment="lab"). The configuration was validated using promtool before we ran the service.

```
promtool.exe check config prometheus.yml
```

```
job_name: "windows-telegraf"
scrape_interval: 10s
static_configs:
  - targets: ["127.0.0.1:9273"]
    labels:
      os: "windows"
      environment: "lab"
```

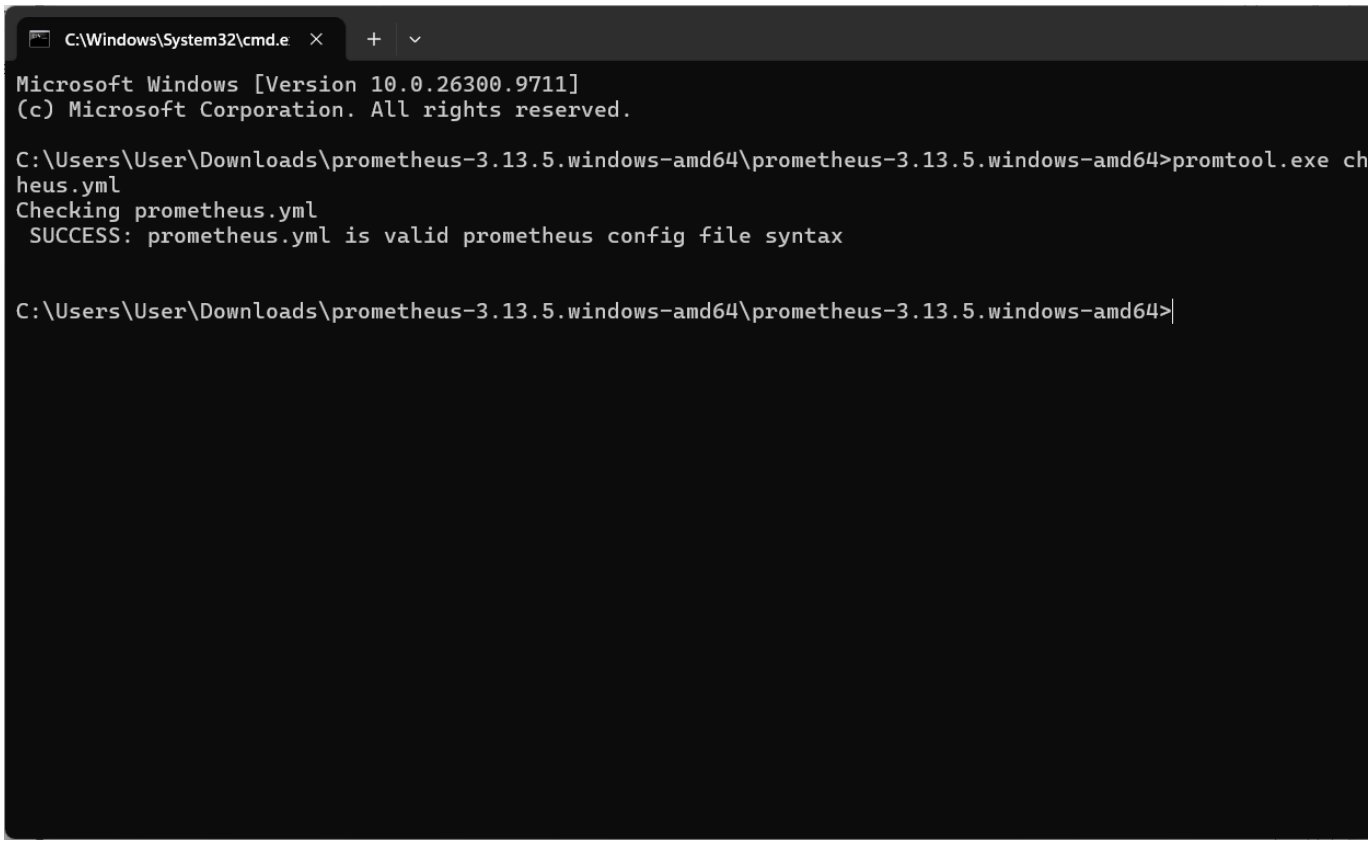


Figure 4. Command Prompt: promtool confirms that prometheus.yml is a valid configuration.

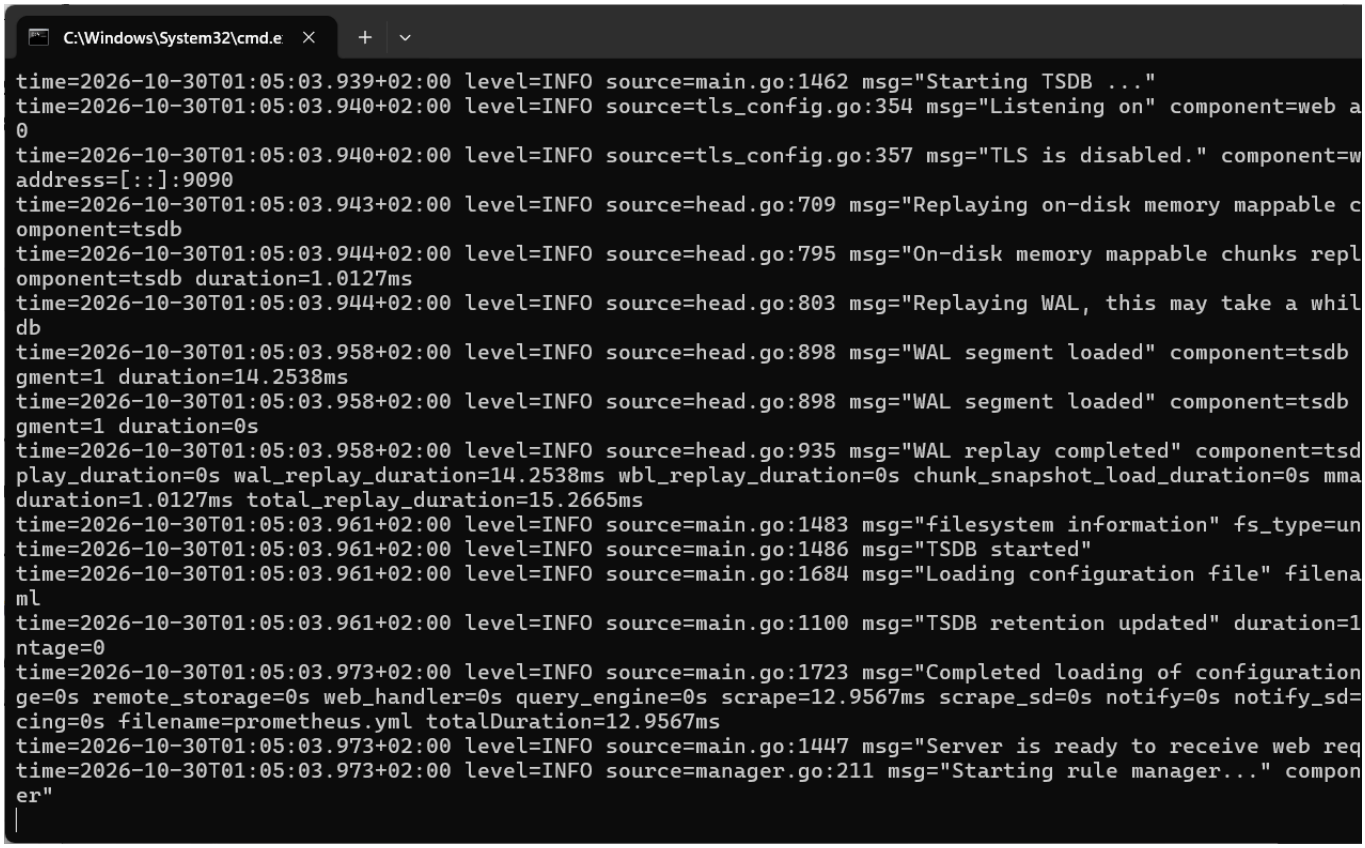


Figure 5. Command Prompt: Prometheus service startup logs indicate the TSDB has started and the server is ready.

The Target health page (Figure 1) confirms independently that the process running was able to scrape both endpoints

4.3 Grafana data source and monitoring panels

Step 7: In Grafana, I connected Grafana to Prometheus using its local HTTP endpoint and verified that the data source was working.

Step 8: I created a new dashboard and added individual Time Series panels. I used the code editor available under the PromQL menu to write the below queries. After each query returned data, I gave the panel a descriptive name, added units and a legend before saving.

I had connected Grafana to Prometheus through the Local HTTP URL. The dashboard had been created with the title Centralized System & Network Monitoring. I had added 4 time series panels. I had set the dashboard to give a view of the data over the last 15 minutes. I had also left the dashboard set to refresh automatically at 10 seconds. The values will be scraped and stored by Prometheus. The refresh rate will dictate how frequently they appear on the Grafana page.

Panel	PromQL query / metric	Display configuration
CPU usage	100 - cpu_usage_idle{cpu="cpu-total",job="windows-telegraf"}	Percent 0–100, legend Windows CPU
RAM usage	mem_used_percent{job="windows-telegraf"}	Percent 0–100, legend Windows RAM
Disk usage	disk_used_percent{job="windows-telegraf"}	Percent 0–100, legend {{device}}
Network download	rate(net_bytes_recv{job="windows-telegraf"}[1m])	bytes/sec (SI), Download - {{interface}}
Network upload	rate(net_bytes_sent{job="windows-telegraf"}[1m])	bytes/sec (SI), Upload - {{interface}}

The full 4 panel view of the dashboard is presented on the front cover. Each panel shows individual metrics from the dashboard below.

5. Implementation Process and Observations

5.1 CPU and memory panels

This is calculated by subtracting the overall idle percentage from 100 on the CPU panel. The Memory panel shows how much RAM is being used. We saw some variance in the CPU graph during the monitored sessions. The memory was usually between 75-83% used. The data shown in this monitoring tool only reflects the performance of the host at the time the data is captured.

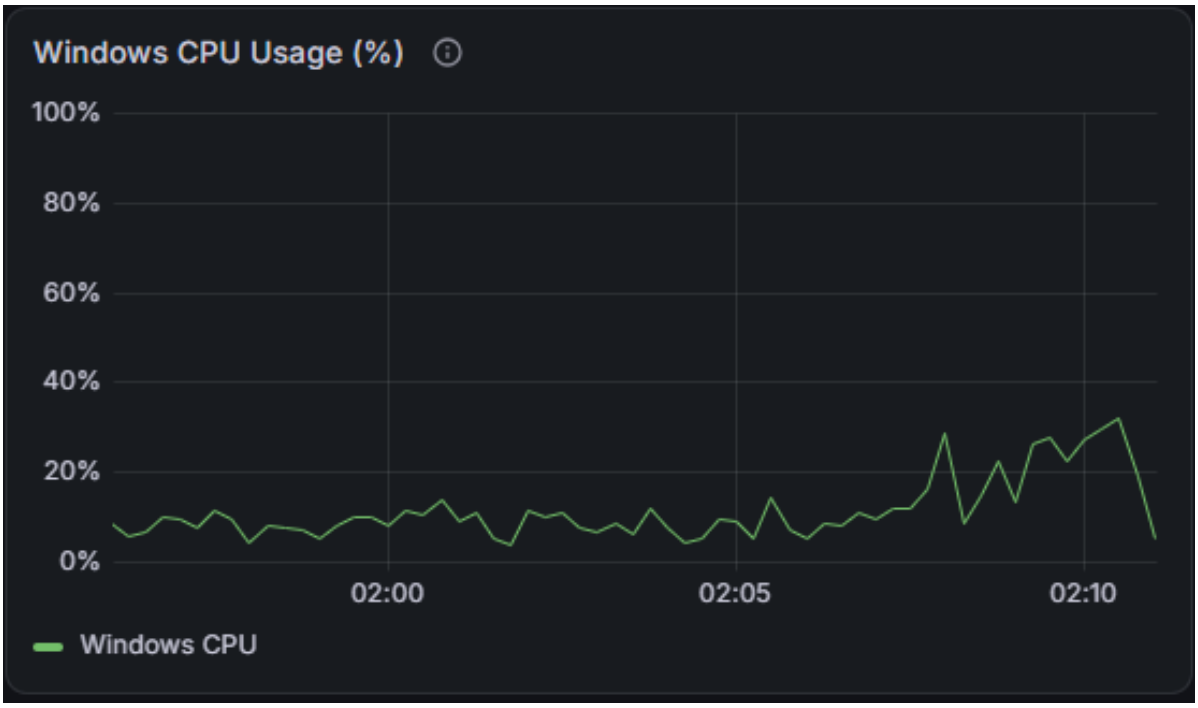


Figure 6. CPU panel showing live utilization over a 15-minute window.

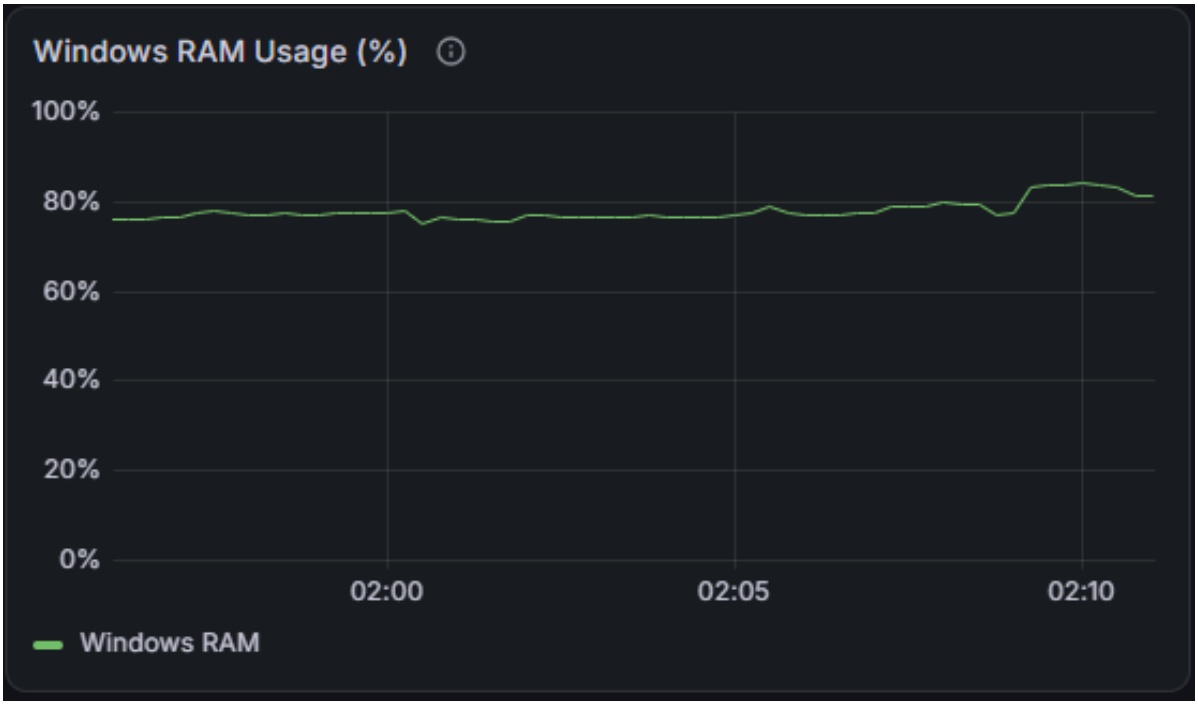


Figure 7. Memory utilization displayed in the Grafana dashboard.

5.2 Disk utilization panel

We can see the disks shown with the Telegraf disk_used_percent metric. The custom legend {{device}} has abbreviated the volume names to just show C: and D:. The usage of D: is around 75% whilst C: is almost 100%. This is why we have a high disk usage alert, to check there is a legitimate issue with disk space.

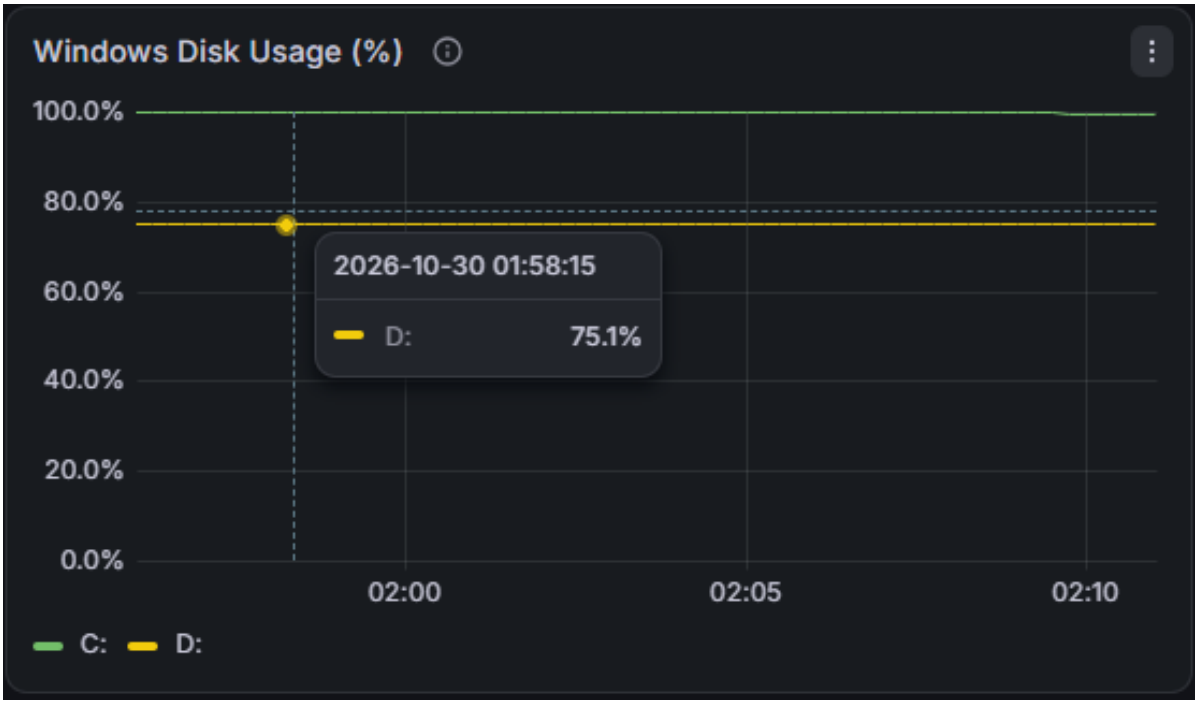


Figure 8. Disk usage panel distinguishing C: and D: drives.

5.3 Network monitoring panel

This can be seen below where we have queried for download and upload rates on the network, using the rate of the total bytes received and sent. These can involve multiple interfaces so we have added custom legends with the interface names. We can see in our visualizations that both ethernet and Wi-Fi have been used. We have specified the data rate is in bytes/sec (SI) meaning Grafana will format them into kB/s. During the live demo there was normal browsing within the browser to show how the curves change.

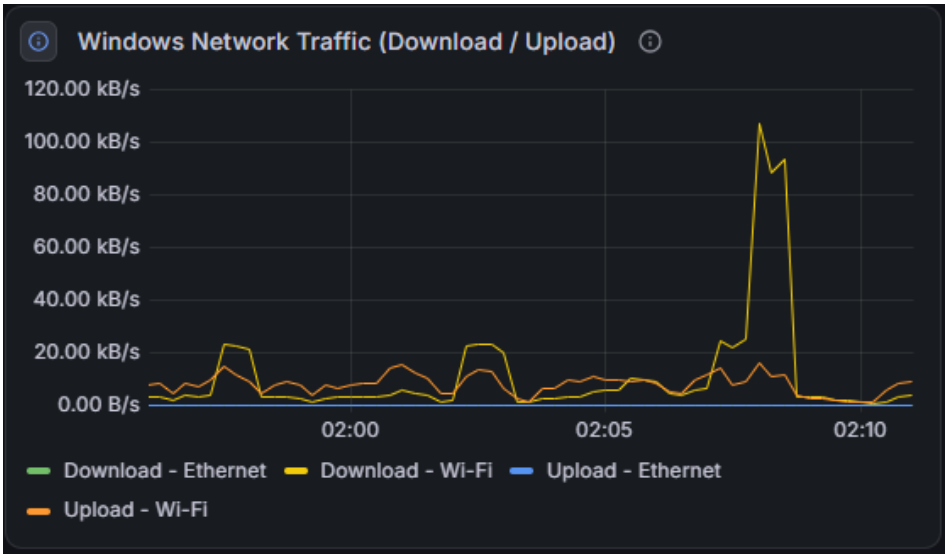


Figure 9. Network transmit and receive time series with interface-aware legend entries.

6. Grafana Alert Rule Configuration

Step 9: Create a new rule under the 'Windows Monitoring Alerts' folder. We have applied the exact time series from the C: drive by filtering on the device label and set our alert condition to be IS ABOVE 90.

Step 10: Apply the 'Windows Monitoring' evaluation group to evaluate the rule once every minute. Set the evaluation group to evaluate once every minute and to wait once before firing. We have selected the empty contact point as we want to view the alert within the product. We have not enabled any external notifications.

Step 11: Our rule will now show as breached within the preview. Our rule will show as 'Normal' when saved but will change to 'Firing' as it has been scheduled. Within the alert rules list we can see our active rule.

We have created an alert called 'Windows C Drive - High Disk Usage'. This will highlight if there is a risk of running out of space. This rule has been created using Grafana. We have used an instant Prometheus query to check that the value from the C: volume is greater than 90. This has been saved under the 'Windows Monitoring Alerts' folder. We have applied the 'Windows Monitoring' evaluation group. The rule will evaluate once every minute and will wait once before firing. We have set the keep firing for to 0 seconds. This means that the condition must be breached for long enough for the rule to evaluate and change to 'Firing'.

```
disk_used_percent{job="windows-telegraf",device="C:"} > 90
```

Setting	Configured value
Rule name	Windows C Drive - High Disk Usage
Threshold	IS ABOVE 90
Evaluation group	Windows Monitoring
Evaluation interval	1 minute
Pending period	1 minute
Keep firing for	0 seconds
Contact point	empty (no outbound notifications)

First we can see that the threshold was breached. However when looking at the alert it first appears as Normal, but once it has been assessed it changes to Firing. This shows us that the preview is a test alert and the rule list shows us what the real state is. No email/SMS or other notifications have been set up.

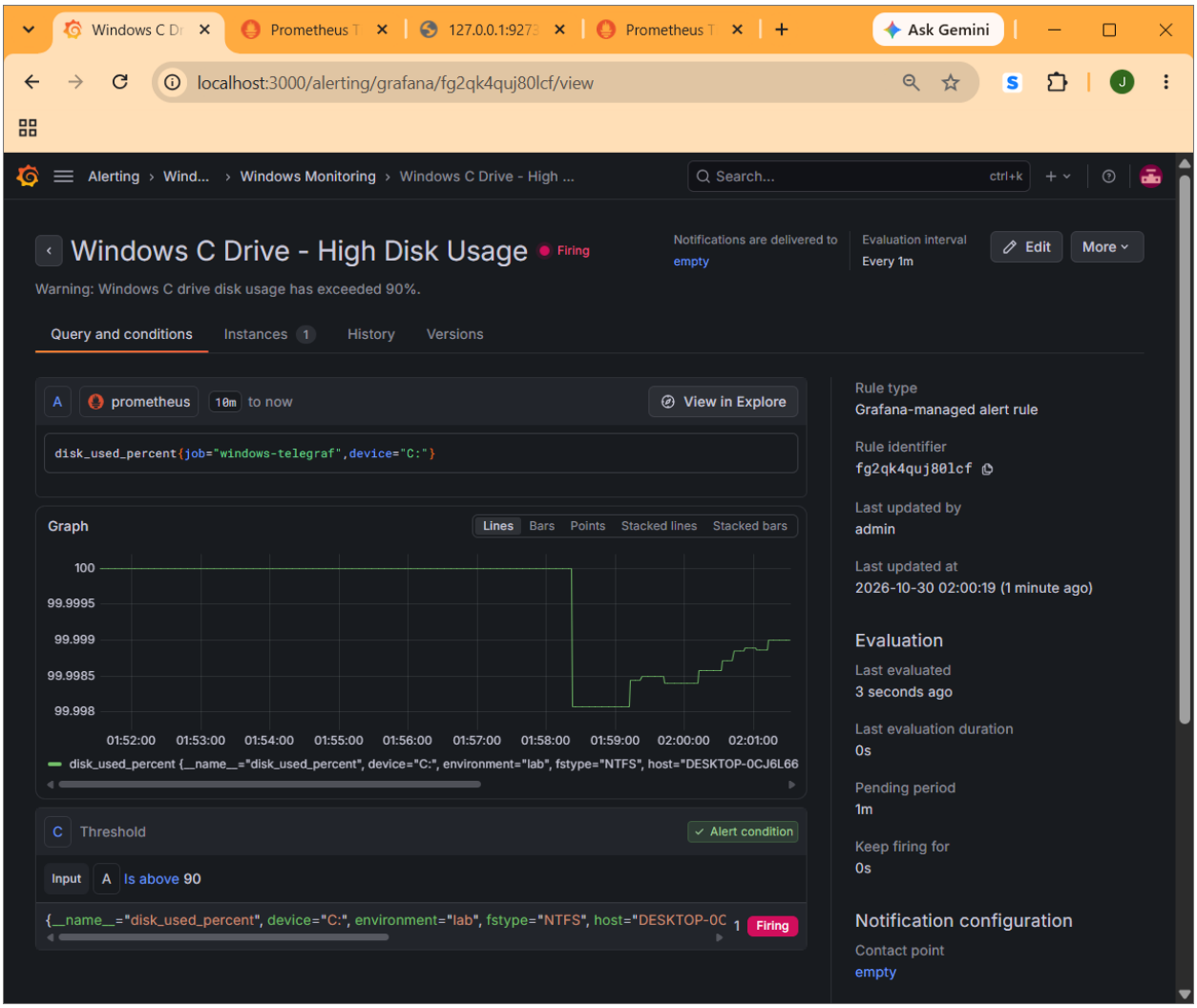


Figure 10. Saved alert rule evaluated as Firing, showing the rule name, threshold and evaluation details.

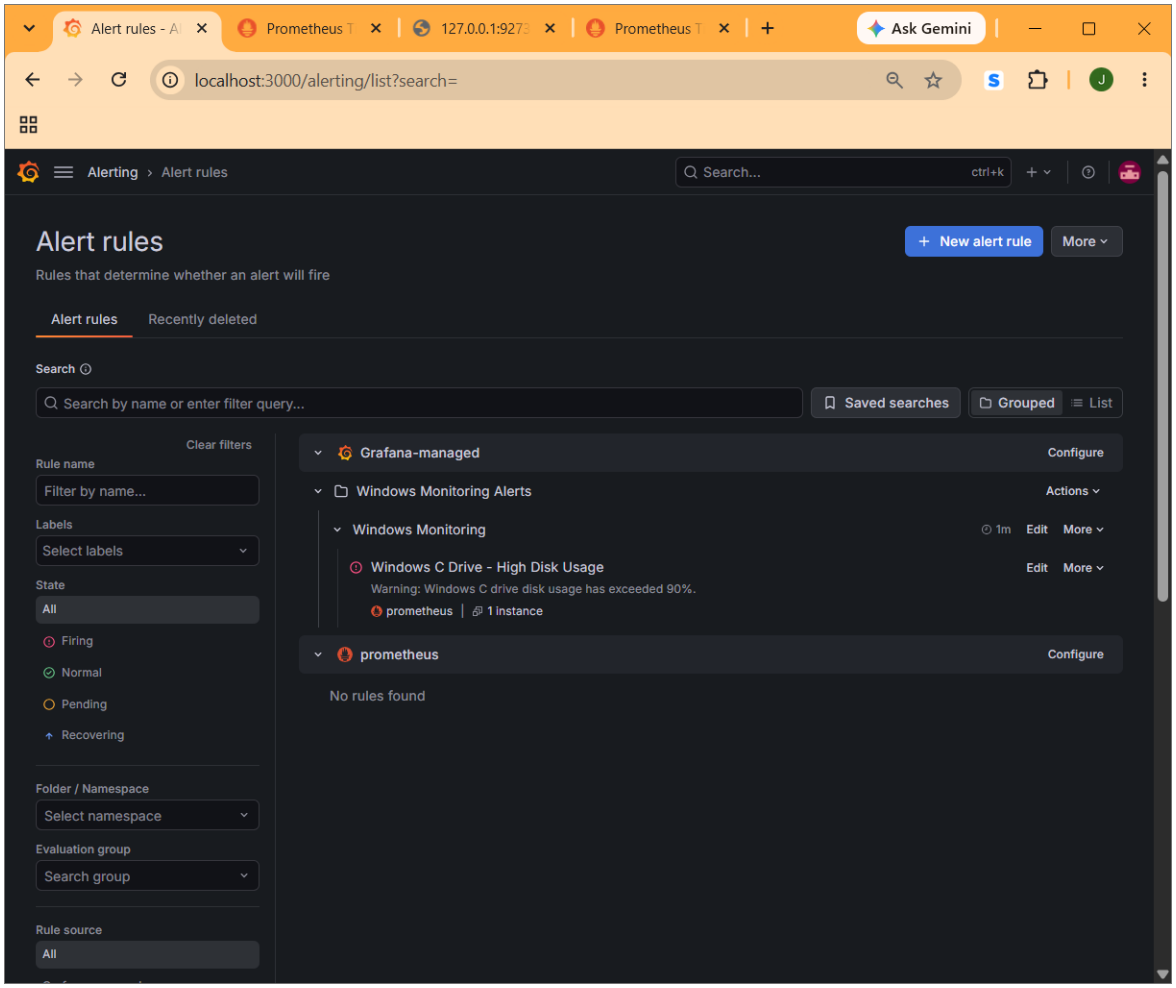


Figure 11. Grafana alert-rules list with the saved Windows disk rule and its alert state.

7. Verification and Live Testing

Test	Method	Observed outcome
Metrics endpoint	Open 127.0.0.1:9273/metrics	Prometheus-formatted metrics displayed
Prometheus job status	Open localhost:9090/targets	prometheus and windows-telegraf both UP
CPU and RAM visualization	Load Grafana dashboard	Non-static live time-series data displayed
Disk volume differentiation	Check disk legend	C: and D: shown separately
Network capture	Observe rate queries during ordinary browser use	Receive/transmit values varied in time
Alert threshold	Inspect saved alert rule after evaluation	High disk usage alert reached Firing
Auto refresh	Save 10s refresh; reload dashboard	10s still selected after reload
Backup	Copy dashboard as resource JSON	JSON backup created; not a full database backup

A 3-and-a-half minute video was taken as evidence. This demonstrates real time information being shown on the dashboards. Both Prometheus targets are showing UP and the alerts section. This supports the screenshots but does not replace validation the configurations.

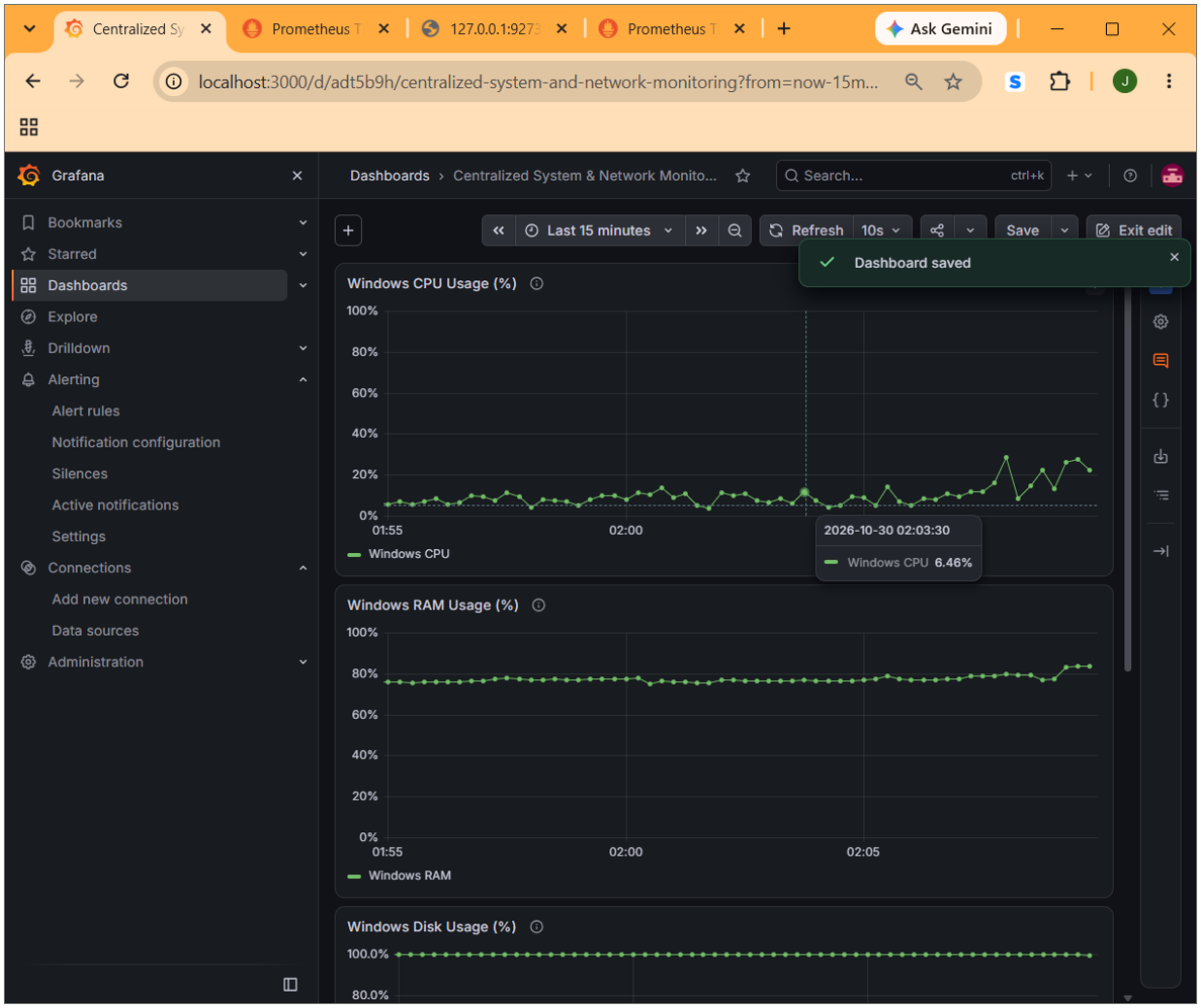


Figure 12. Save confirmation showing the dashboard was written successfully.

The JSON dashboard resource was copied to a .json file to serve as an additional backup of the saved dashboard.

8. Issues Encountered and Resolutions

8.1 Initial collector and endpoint setup

An initial attempt was made to use windows_exporter but ran into issues with Windows' security software quarantining the tool and some issues with installing the service. The final setup does not use this exporter, but rather uses Telegraf to export the metrics from the host. This means that the final health of the service and dashboards will be done using Telegraf.

8.2 Dashboard save failure caused by disk pressure

The C: drive of the computer was running out of space, leading to the error "database or disk is full (13)" appearing in Grafana when attempting to save the dashboard refresh setting. Saving again later gave no error and the dashboard updated every 10 seconds. When recording again, there was around 1.85 GB free on the C: drive and 3.85GB free on the D: drive. It was not identified why it was sometimes possible to save after the error occurred, however, having low space on the drive means it could cause issues.

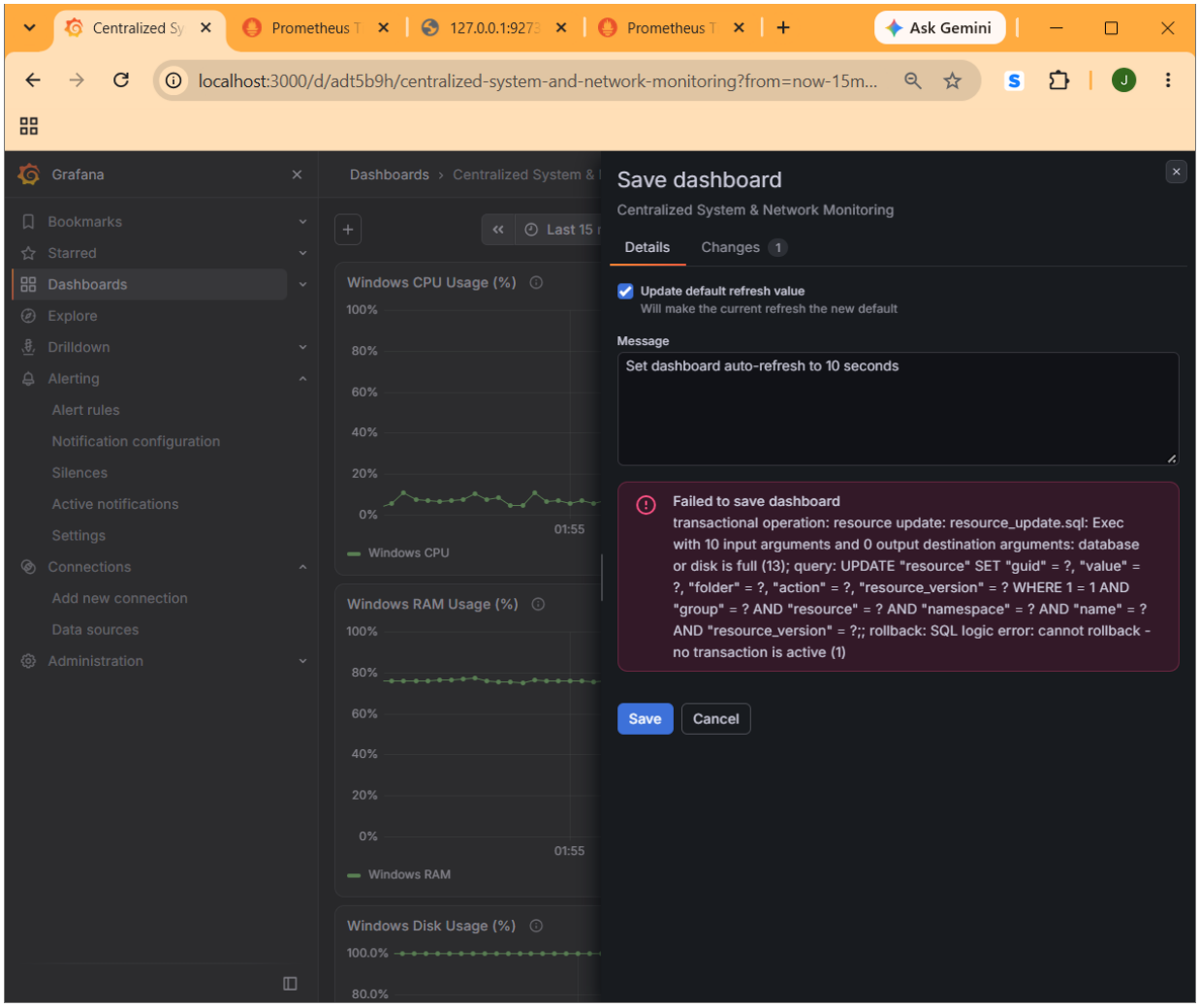


Figure 13. Grafana reported a database-or-disk-full error while saving the refresh default.

A Windows This PC free space check indicated that there is approximately 1.85GB of free space on the C: drive and 3.85GB of free space on the D: drive. This does indicate that there is not much free space left on the C: drive which can cause issues with reliability.

9. Scope, Limitations and Security Considerations

- The completed production dashboard monitors one Windows host, not an enterprise fleet or multi-node network.
- Only one Windows workstation was configured as a monitored target for the delivered system.
- Grafana plots summarized interface byte counters, rather than packet captures or application-layer traffic inspection.
- The empty contact point permits local alert-state demonstration but does not deliver remote notifications.
- The dashboard JSON backup saves dashboard resource configuration, but not Prometheus history or Grafana alert rules.
- The Windows host should maintain additional disk free space for ongoing monitoring database growth and stable operation.

10. Future Work

In future versions further hosts can be monitored by having more exporters on the network and more detailed service availability monitoring. A central contact point can be created so that alerts can be sent to an administrator. Alerting can also be adjusted to suit the workload of the machine.

Further work can also be done to include using Containerlab. This would allow a virtual network topology to be created with compatible router/switch/network appliance images. These virtual devices can be set to expose metrics that are compatible with Prometheus. The dashboard can then be viewed using Grafana. This allows the whole topology to be monitored. This is proposed work for future versions and was not implemented or tested in this deliverable.

11. Conclusion

This project has created a working local monitoring solution using the tools Telegraf, Prometheus & Grafana. There are 4 panels on the dashboard showing real data relating to CPU, Memory, Disk and Interface Traffic. Prometheus shows that all of the scrape targets are healthy. The disk alert has been set up in Grafana and shows as Firing when the C: Drive has not dropped below the set 90% threshold. This shows that the solution is working as the dashboard is monitoring, there is evidence of alerts and everything has been recorded live. The next steps for this project will be to improve the disk headroom, set up real alerting and expand the design to monitor more networked machines.

12. Presentation & Live Demonstration

The PowerPoint presentation, including the recorded live demonstration of the monitoring system, is available through the following link:

View PowerPoint Presentation & Live Demo

References

[1] Grafana Labs. Grafana documentation. https://grafana.com/docs/grafana/latest/

[2] Prometheus Authors. Prometheus documentation. https://prometheus.io/docs/

[3] InfluxData. Telegraf documentation. https://docs.influxdata.com/telegraf/

[4] Containerlab. Documentation and examples. https://containerlab.dev/

[5] AuthorGate. Systems and Networks Administration submission repository. https://github.com/AuthorGate/systems-and-networks-administration